

Projeto 6 Cobol – Semana 8

Sistema de Contas Bancárias

Vamos continuar o projeto anterior e colocar em prática tudo que aprendemos até agora, com foco no aprendizado recente de banco de dados.

Cenário

Um banco precisa processar diariamente um arquivo contendo transações de débito e crédito dos clientes. As informações dos clientes são armazenadas em tabelas DB2 e devem ser mantidas atualizadas após o processamento das transações.

O sistema deverá processar os arquivos de entrada, atualizar os registros do banco de dados, gerar arquivos de saída, relatórios e estatísticas da execução.

Requisitos

Desenvolver um sistema BATCH em COBOL que:

- 1 – Ordena os arquivos de entrada;
- 2 – Processa as regras necessárias;
- 3 – Atualiza as informações no DB2;
- 4 – Gera:
 - Relatórios;
 - Arquivo de saída (com status dos registros);
 - Logs de processamento

Fluxo:

Arquivo de entrada -> Programa(s) COBOL -> DB2 -> Saídas

Criar copybooks de entrada de acordo com as colunas existentes nas tabelas;
Arquivo de saída deverá exibir o log do processamento, com sucesso ou erro para cada cliente.

Regras de negócio:

- Não cadastrar cliente duplicado;
- Nome é obrigatório;
- Na movimentação de saque, verificar se o saldo é suficiente;
- Depósito sempre será permitido;
- Atualizar o saldo no DB2.

Modelo de Banco (DB2)**Tabela CLIENTES**

```
CREATE TABLE CLIENTES (  
  CLI_ID INTEGER NOT NULL,  
  CLI_NOME VARCHAR(30) NOT NULL,  
  CLI_SALDO DECIMAL(9,0) NOT NULL,  
  DT_ATUALIZACAO DATE,  
  PRIMARY KEY (CLI_ID)  
);
```

Tabela TRANSAÇÕES

```
CREATE TABLE TRANSACOES (  
  TRX_ID INTEGER NOT NULL,  
  CLI_ID INTEGER NOT NULL,  
  TRX_TIPO CHAR(1) NOT NULL,  
  TRX_VALOR DECIMAL(9,0) NOT NULL,  
  DT_PROCESSAMENTO DATE,  
  PRIMARY KEY (TRX_ID)  
);
```

Tabela ERROS

```
CREATE TABLE ERROS_PROCESSAMENTO (  

```



Classificação de confidencialidade: Restrito

```
ID_ERRO INTEGER GENERATED ALWAYS AS IDENTITY,  
CLI_ID INTEGER,  
DESCRICAO_ERRO VARCHAR(100),  
DT_OCORRENCIA TIMESTAMP  
);
```

Exemplo de arquivos de entrada:

CLIENTES.TXT

```
00123JOAO SILVA 000010000  
00456MARIA SOUZA 000025000  
00789CARLOS PEREIRA 000005000
```

TRANSACOES.TXT

```
0012300010C000000500  
0012300020D000000200  
0045600030D000001000
```

Lógica do processamento batch:

Criar uma rotina JCL que execute:

- Leitura do arquivo clientes;
- Verificar se o cliente já existe na tabela CLIENTES. Se não existir, inserir. Se existir, atualizar nome e saldo;
- Leitura do arquivo transações;
- Para cada registro do arquivo, validar: cliente existente, tipo da transação, valor maior que zero, saldo suficiente para débito;
- Inserir a transação na tabela;
- Atualizar saldo;
- Gerar relatórios.

Relatórios:

- Relatório de processamento:
Total lidos, total processados, total com erro;
- Relatório detalhado:
Cliente, operação, status.

Log de processamento:

- Quantidade de registros;
- Erros encontrados;
- Mensagens de falha de DB2.

Seguir a organização do código.

Realizar tratamento de erros e exibir logs de erro.

Quando ocorrer erro:

- Registrar no arquivo ERROS.TXT;
- Inserir registro na tabela de erros.
- Erros obrigatórios: Cliente inexistente, tipo inválido, valor zerado, saldo insuficiente.

Controle transacional:

- Implementar COMMIT a cada 100 registros processados;
- ROLLBACK em caso de erro SQL;
- Tratamento dos SQLCODES principais

Você deverá executar o JCL criado via TSO e enviar prints mostrando a execução do programa com sucesso.

Desafios extras (opcional)

Se desejar subir o nível do seu programa:

- Utilizar cursor DB2 para geração do relatório final;
- Implementar tabela de histórico de transações;
- Criar índices para otimização;
- Gerar relatório final consultando diretamente o DB2.

Critérios de Avaliação

1. **Funcionamento:** As funcionalidades devem operar conforme especificado.
2. **Organização do Código:** Código organizado, seguindo a estrutura proposta.
3. **Interatividade:** A interação com o usuário deve ser clara e amigável.